```
#define TRUE  1
#define FALSE 0

#define LED1 PORTL6         //Connected to PORTL
#define LED2 PORTL7         //Connected to PORTL

//Address ranges from 0 - 4095 for 24C32 and 0 - 8191 for 24C64 chips
void EEOpen();
char EEWriteByte(unsigned int address, unsigned char data);
unsigned char EEReadByte(unsigned int address);

void USART0_Init (unsigned int baud);
void USART0_Transmit (unsigned char data);
unsigned char USART0_Receive (void);
void USART0_Receive_String(char *s, int length);

char arr[4096];

__C_task main(void)
{
        unsigned int address;

        //Initialize USART1 at 115.2kbps with 16MHz Crystal
        USART0_Init(8);
        SETBIT(DDRL, PL6);  //Set data direction registers for LEDs
        SETBIT(DDRL, PL7);
        while(USART0_Receive() != 'A');
        USART0_Transmit('W');
        EEOpen();
        //Fill whole eeprom 8KB (8096 bytes) with incomming data over USART
        USART0_Receive_String(arr, 4096);
//      for(address=0;address<255;address++)
  //                    USART0_Transmit(arr[address]);

        char failed=0;
        for(address=0;address<4096;address++)
        {
                if(EEWriteByte(address, arr[address])==0)
                {
                                        //Write Failed
SETBIT(PORTL, LED1);
                        failed=1;
                        break;
                }
        }
        if(!failed)
         SETBIT(PORTL, LED2);
        failed=0;
        for(address=0;address<4096;address++)
                        USART0_Transmit(EEReadByte(address));
}
void EEOpen(void)
{
       //Refer: Atmel Application Note, AVR315
        //Set up TWI Module; Setting for 400kHz with 16Mhz Osc.
        TWBR = 12;
                                TWSR &= (~((1<<TWPS1)|(1<<TWPS0)));
}
char EEWriteByte(unsigned int address,unsigned char data)
{
        do
        {
                //Send START Condition
                TCCR=(1<<TWINT)|(1<<TWSTA)|(1<<TWEN);
                //Wait for TWINT flag set
                while(!(TWCR & (1<<TWINT)));
                if((STATUS != START) && (STATUS != REP_START))
                                return FALSE;
                //Write SLA+W to EEPROM @ 00h
                TWDR = 0xa0;
//1010 000 0
                //Initiate Transfer
TWCR=(1<<TWINT)|(1<<TWEN);
                //Wait Till Done
                while(!(TWCR & (1<<TWINT)));
                if(STATUS != MT_SLA_ACK)
                        return FALSE;
        }
        while(STATUS != MT_SLA_ACK);
        //Write ADDRH
        TWDR=(address >> 8);
        //Initiate Transfer
        TWCR=(1<<TWINT)|(1<<TWEN);
        //Wait Till Done
        while(!(TWCR & (1<<TWINT)));
        if(STATUS != MT_DATA_ACK)
                return FALSE;

        //Write ADDRL
        TWDR= address;
        //Initiate Transfer
        TWCR=(1<<TWINT)|(1<<TWEN);
        //Wait Till Done
        while(!(TWCR & (1<<TWINT)));
        if(STATUS != MT_DATA_ACK)
                return FALSE;

        //Write DATA
        TWDR=data;
        //Initiate Transfer
        TWCR=(1<<TWINT)|(1<<TWEN);
        //Wait Till Done
        while(!(TWCR & (1<<TWINT)));
        if(STATUS != MT_DATA_ACK)
                return FALSE;

        //Put Stop Condition
        TWCR=(1<<TWINT)|(1<<TWEN)|(1<<TWSTO);
        //Wait for STOP to finish
        while(TWCR & (1<<TWSTO));
        //Wait untill Writing is complete
        __delay_cycles(34000);
//2.125mS failed for 2mS
        //Return TRUE on Success
        return TRUE;
}
unsigned char EEReadByte(unsigned int address)
{
        unsigned char data;
        //Initiate a Dummy Write Sequence to start Random Read
        do
        {
                //Put Start Condition on TWI Bus
                TWCR=(1<<TWINT)|(1<<TWSTA)|(1<<TWEN);
                //Wait Till Done
                while(!(TWCR & (1<<TWINT)));
                if((STATUS != START) && (STATUS != REP_START))
                        return FALSE;
                //Write SLA+W to EEPROM @ 00h
                TWDR = 0xa0;
//1010 000 0;
                //Initiate Transfer
TWCR=(1<<TWINT)|(1<<TWEN);
                //Wait Till Done
                while(!(TWCR & (1<<TWINT)));
        }
        while(STATUS != MT_SLA_ACK);
        //Write ADDRH
        TWDR=(address >> 8);
        //Initiate Transfer
        TWCR=(1<<TWINT)|(1<<TWEN);
        //Wait Till Done
        while(!(TWCR & (1<<TWINT)));
        if(STATUS != MT_DATA_ACK)
                return FALSE;

        //Write ADDRL
        TWDR= address;
        //Initiate Transfer
        TWCR=(1<<TWINT)|(1<<TWEN);
        //Wait Till Done
        while(!(TWCR & (1<<TWINT)));
        if(STATUS != MT_DATA_ACK)
                return FALSE;

        //*********************** End dummy write sequence **********************
        //Put Start Condition on TWI Bus
        TWCR=(1<<TWINT)|(1<<TWSTA)|(1<<TWEN);
        //Wait Till Done
        while(!(TWCR & (1<<TWINT)));
        if(STATUS != REP_START)
                return FALSE;
        //Write SLA+R to EEPROM @ 00h
        TWDR=0xa1;                  //1010 000 1
        //Initiate Transfer
        TWCR=(1<<TWINT)|(1<<TWEN);
        //Wait Till Done
        while(!(TWCR & (1<<TWINT)));
        if(STATUS != MR_SLA_ACK)
                return FALSE;
        //Enable Reception of data by clearing TWINT
        TWCR=(1<<TWINT)|(1<<TWEN);
        //Wait till done
        while(!(TWCR & (1<<TWINT)));
        if(STATUS != MR_DATA_NACK)
                return FALSE;
        //Read the data
        data=TWDR;
        //Put Stop Condition
        TWCR=(1<<TWINT)|(1<<TWEN)|(1<<TWSTO);
        //Wait for STOP to finish
        while(TWCR & (1<<TWSTO));
        //Return TRUE
        return data;
}
//Initialize USART1
void USART0_Init(unsigned int baud)
{
  /* Set baud rate */
  UBRR0H = (unsigned char)(baud >> 8);
  UBRR0L = (unsigned char)(baud);
  /* Enable receiver and transmitter */
  UCSR0B = (1<<RXEN0)|(1<<TXEN0);
  /* Set frame format: Asynchronous 8 data, 1 stop bit */
  UCSR0C = (1<<UCSZ01)|(1<<UCSZ00);
}

//Read and Write functions for USART0
void USART0_Transmit (unsigned char data)
{
  /* Wait for empty transmit buffer */
  while (!CHECKBIT(UCSR0A, UDRE0));
   /* Put data into buffer, transmits the data */
  UDR0 = data;
}

unsigned char USART0_Receive (void)
{
  /* Wait for data to be received */
  while (!CHECKBIT(UCSR0A, RXC0));
   /* Get and return received data from buffer */
  return UDR0;
}
void USART0_Receive_String(char *s, int length)
{
  unsigned char character;
  character = USART0_Receive();
  while (length--)
  {
    *s++ = character;
    character = USART0_Receive();
  }
}
```